

Der Tomaten-Bewässerungswächter

Bildgestützte Welke-Erkennung mit solarbetriebener ESP32-CAM und Modbus
TCP – Aufbau, Funktionsweise, Datenübertragung und Bedienung

Prof. Dr.-Ing. Ralph Wystup

7. August 2026

Inhaltsverzeichnis

1 Überblick: Was das System leistet	3
1.1 Beteiligte Komponenten	3
1.2 Datenfluss	3
2 Das Zusammenspiel der Programme	4
2.1 Rollenverteilung	4
2.2 Der Registerplan als gemeinsame Sprache	4
2.3 Die drei Schleifen	5
3 Der Bild-Download im Detail	5
3.1 Ablauf einer Bildübertragung	5
3.2 Datenraten: was zu erwarten ist	6
3.3 Warum überhaupt Modbus?	6
4 Namensvergabe: die Kamera ohne feste IP finden	7
4.1 Das Problem	7
4.2 Die Lösung: zwei Namen, ein Gerät	7
5 Die Stromabschaltung: Erkennung und sicherer Wiederanlauf	7
5.1 Was bei der Abschaltung passiert	7
5.2 Wie das Dashboard den Ausfall erlebt	8
5.3 Der Wiederanlauf, Schritt für Schritt	8
5.4 Zeitstempel: warum der PC stempelt und nicht die Kamera	8
5.5 Software-Updates ohne Kabel (OTA)	9
6 Die Bildauswertung: von Pixeln zur Gieß-Empfehlung	9
6.1 Schritt 1: Die Pflanze vom Hintergrund trennen (Excess-Green-Index)	9
6.2 Schritt 2: Zwei Kennwerte je Bild	9
6.3 Schritt 3: Glättung	10
6.4 Schritt 4: Vergleich mit der Referenz und Ampel	10
6.5 Grenzen des Verfahrens	11
7 Bedienungsanleitung	11
7.1 Einmalige Einrichtung	11
7.1.1 Kamera flashen	11
7.1.2 PC vorbereiten	11
7.1.3 Montage	11
7.2 Täglicher Betrieb	12
7.2.1 Starten	12

7.2.2	Referenz setzen – der wichtigste Handgriff	12
7.2.3	Die Dashboard-Elemente	12
7.2.4	Dateien, die das Programm anlegt	12
7.3	Software-Update der Kamera (über WLAN)	13
7.4	Fehlerbehebung	13
7.5	Einstellparameter im Überblick	13

8	Zusammenfassung	14
----------	------------------------	-----------

1 Überblick: Was das System leistet

Eine ESP32-CAM beobachtet eine Tomatenpflanze. Ein Python-Programm auf dem PC holt in festem Takt Bilder von der Kamera, trennt die Pflanze rechnerisch vom Hintergrund und bestimmt zwei Kennwerte: die sichtbare **Grünfläche** und den **Schwerpunkt** der Blattmasse im Bild. Welkende Blätter hängen durch – die Grünfläche schrumpft und der Schwerpunkt sinkt ab. Beide Kennwerte werden mit einer Referenz verglichen, die einmal direkt nach dem Gießen gesetzt wird. Daraus entsteht eine einfache Ampel:

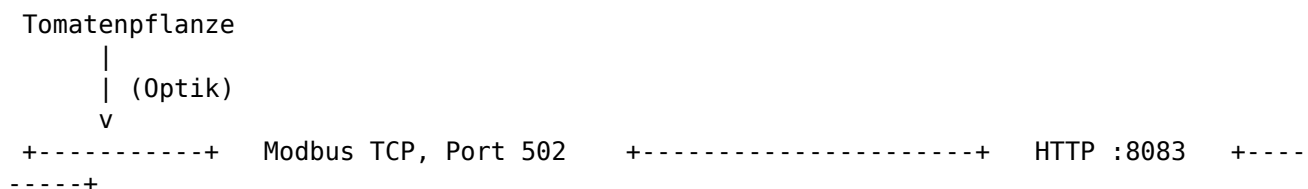
Ampel	Bedeutung
GRÜN	Wasser ausreichend
GELB	beobachten – Pflanze wird schlapp
ROT (blinkend)	GIESSEN! Blätter hängen deutlich
GRAU	keine Aussage möglich (Nacht, Pflanze nicht erkannt, Referenz fehlt)

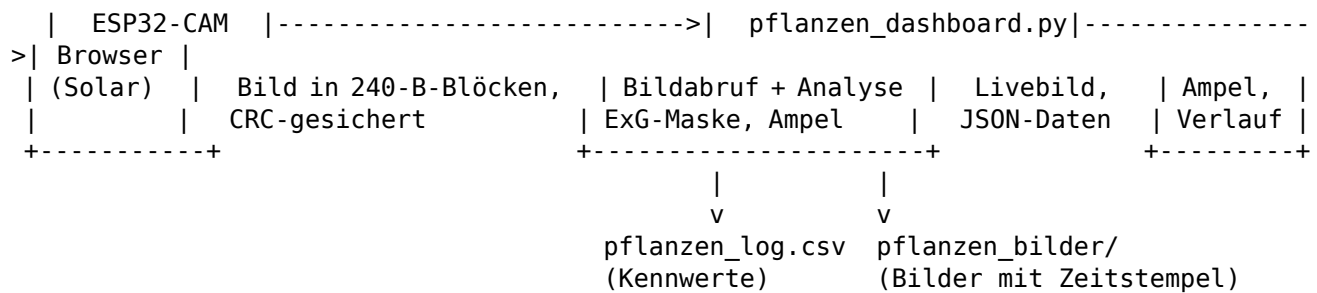
Die Besonderheit der Anlage: Die Kamera hängt **im Freien an einem Solarmodul**, dessen Laderegler die Versorgung bei 10 % Akkuladung hart abschaltet und erst bei 50 % wieder einschaltet. Die Kamera wird also regelmäßig stromlos und startet neu – mit allen Konsequenzen für IP-Adresse, Bildzählung und Gerätezustand. Das System ist so ausgelegt, dass es diese Zyklen ohne jeden Handgriff übersteht (Kapitel 6 und 7).

1.1 Beteiligte Komponenten

Komponente	Aufgabe
ESP32-CAM (AI Thinker)	nimmt auf Anforderung ein JPEG auf und stellt es als Modbus-TCP-Slave blockweise in Holding-Registern bereit
Solarmodul mit Laderegler	versorgt die Kamera; Abschaltung bei 10 %, Wiedereinschaltung bei 50 % Akkuladung
ESP32_CAM_Modbus_Snapshot.ino	Arduino-Sketch der Kamera: Kamera, WLAN, Modbus-TCP-Server, Statusregister, OTA-Update
pflanzen_dashboard.py	Python-Programm auf dem PC: Bildabruf, Auswertung, Ampel, Web-Oberfläche (Port 8083), Protokollierung
Fritz!Box	WLAN und Namensauflösung (tomatencam.fritz.box)
Browser	Anzeige des Dashboards unter http://localhost:8083

1.2 Datenfluss





Wichtig für das Verständnis: Die Kamera ist ein reiner **Diener** (Slave). Sie sendet nie von sich aus, sondern beantwortet nur Modbus-Anfragen des PC-Programms. Das PC-Programm ist der **Meister** (Master) und bestimmt den Takt. Der Browser wiederum fragt nur das PC-Programm ab, nie die Kamera – zur Kamera besteht stets **genau eine** TCP-Verbindung.

2 Das Zusammenspiel der Programme

2.1 Rollenverteilung

Der Arduino-Sketch auf der Kamera macht bewusst so wenig wie möglich: Bild aufnehmen, in Register packen, Fragen beantworten. Alles “Kluge” – Analyse, Bewertung, Speicherung, Anzeige – liegt auf dem PC. Das hat drei Gründe:

1. **Robustheit.** Je weniger der Mikrocontroller tut, desto weniger kann nach einem harten Stromausfall in undefiniertem Zustand hängen bleiben.
2. **Rechenleistung.** Die Bildauswertung mit numpy ist auf dem PC eine Sache von Millisekunden; auf dem ESP32 wäre sie mühsam und speicherkritisch.
3. **Wartbarkeit.** Die Auswertung lässt sich am PC ändern und testen, ohne die Kamera neu zu flashen.

2.2 Der Registerplan als gemeinsame Sprache

Beide Programme kennen denselben Registerplan – er ist die vollständige Schnittstellenbeschreibung des Systems:

Register	Inhalt	Zugriff
0	Bildnummer (zählt je Aufnahme hoch, Reset bei Neustart!)	lesen
1	Status: 0 = kein Bild, 2 = Bild bereit, 3 = Kamerafehler	lesen
2/3	Bildgröße in Bytes (High-/Low-Word, 32 Bit)	lesen
4	Blockgröße in Bytes (fest 240)	lesen
5	Blockanzahl des aktuellen Bildes	lesen
6	Blockindex: welcher Block in 100..219 liegt	lesen + schreiben
7	reserviert	–
8	Kommando: 1 schreiben = neue Aufnahme auslösen	schreiben
9	Blitz-LED (GPIO 4): 0/1	lesen + schreiben

Register	Inhalt	Zugriff
10	CRC16 über das gesamte JPEG (Polynom 0xA001)	lesen
11/12	Betriebszeit seit Einschalten in Sekunden (High-/Low-Word)	lesen
13	WLAN-Signalstärke (RSSI) in dBm, Zweierkomplement	lesen
14	freier Arbeitsspeicher (Heap) in kB	lesen
99	CRC16 des aktuell eingeblendeten Blocks	lesen
100..219	Blockdaten: 120 Register = 240 Bytes, High-Byte zuerst	lesen

Verwendet werden nur die Modbus-Funktionscodes FC03/FC04 (Register lesen) und FC06 (einzelnes Register schreiben). Die Register 11 bis 14 sind die "Gesundheitsdaten" der Kamera; sie wurden eigens für den Solarbetrieb ergänzt (Kapitel 6).

2.3 Die drei Schleifen

Im Betrieb laufen drei unabhängige Schleifen:

1. **Kamera-Schleife** (`loop()` im Sketch): horcht auf OTA-Updates, nimmt Modbus-Telegramme entgegen, prüft alle 5 s die WLAN-Verbindung und verbindet bei Abriss selbsttätig neu.
2. **Abfrageschleife** (Thread im Python-Programm): holt im eingestellten Intervall (Standard 300 s) ein Bild, liest die Statusregister, wertet aus, protokolliert, speichert. Jeder Fehler führt zum sauberen Verbindungsabbau und einem neuen Versuch im nächsten Takt.
3. **Web-Schleife** (HTTP-Server im Python-Programm): beantwortet die Anfragen des Browsers (/daten als JSON alle 3 s, /bild.jpg, /maske.jpg, /referenz, /blitz). Sie greift nur auf die zuletzt abgelegten Ergebnisse zu und wartet nie auf die Kamera.

Abfrage- und Web-Schleife teilen sich die Daten über ein Schloss (`threading.Lock`), damit der Browser nie ein halb aktualisiertes Bild oder inkonsistente Kennwerte sieht.

3 Der Bild-Download im Detail

3.1 Ablauf einer Bildübertragung

Ein vollständiger Bildabruf besteht aus vier Phasen:

1. **Auslösen:** Der PC schreibt eine 1 in Register 8. Die Kamera verwirft zunächst einen Frame (damit kein veraltetes Bild aus dem Puffer kommt) und nimmt dann auf. Die Modbus-Antwort kommt erst **nach** der Aufnahme – deshalb arbeitet der PC mit 8 s Timeout.
2. **Kopf lesen:** Der PC liest Register 0 bis 10 in einem Rutsch: Bildnummer, Status, Größe, Blockanzahl, Gesamt-CRC.
3. **Blöcke holen:** Für jeden Block $b = 0 \dots n - 1$: Blockindex in Register 6 schreiben, dann Register 99 bis 219 in **einem** FC03-Telegramm lesen (121 Register = Block-CRC + 240 Bytes Nutzdaten). Stimmt die CRC16 des Blocks nicht, wird derselbe Block bis zu fünfmal neu angefordert – nur dieser eine Block, nicht das ganze Bild.
4. **Gesamtprüfung:** Länge vergleichen, CRC16 über das ganze JPEG prüfen, JPEG-Marker kontrollieren (Anfang FF D8, Ende FF D9). Erst wenn alles stimmt, gilt das Bild als gültig.

Diese doppelte CRC-Sicherung (je Block und übers Ganze) stammt aus dem Vorgängerprojekt und hat sich bewährt: Einzelne WLAN-Störungen kosten nur die Wiederholung eines 240-Byte-Blocks statt einer kompletten Übertragung.

3.2 Datenraten: was zu erwarten ist

Je Block sind **zwei** Modbus-Transaktionen nötig (Index schreiben, Daten lesen). Auf Telegrammebene fallen dabei an:

Telegramm	Größe
FC06-Anfrage (Blockindex schreiben)	12 Bytes
FC06-Antwort (Echo)	12 Bytes
FC03-Anfrage (121 Register lesen)	12 Bytes
FC03-Antwort (Kopf + 242 Bytes Daten)	251 Bytes

Für 240 Nutzbytes werden also rund 287 Bytes übertragen – die Protokoll-Effizienz ist mit etwa 84 % ordentlich. Begrenzend ist aber nicht die Datenmenge, sondern die **Umlaufzeit** (Round-Trip-Zeit t_R) im WLAN, denn der Master wartet nach jeder Anfrage auf die Antwort:

$$t_{\text{Block}} \approx 2t_R + t_{\text{Daten}} \quad v \approx \frac{240 \text{ B}}{t_{\text{Block}}}$$

Mit typischen WLAN-Umlaufzeiten von 5 bis 20 ms ergibt das:

t_R	Zeit je Block	effektive Rate	30-kB-Bild (125 Blöcke)
5 ms	ca. 11 ms	ca. 21 kB/s	ca. 1,5 s
10 ms	ca. 21 ms	ca. 11 kB/s	ca. 3 s
20 ms	ca. 41 ms	ca. 6 kB/s	ca. 5–6 s

Hinzu kommen etwa 0,5 bis 1 s für die Aufnahme selbst (Phase 1). In der Praxis sind **einige Sekunden pro VGA-Bild** (640 x 480, JPEG-Qualität 10, typisch 25 bis 40 kB) normal; bei schlechtem WLAN-Empfang draußen am Solarstandort entsprechend mehr. Die tatsächlich erreichte Dauer und Rate zeigt das Dashboard bei jeder Übertragung an (“Dauer”, “kB/s”) – zusammen mit dem RSSI-Wert (Register 13) lässt sich so beurteilen, ob der Standort funktechnisch taugt. Als Faustwerte: RSSI besser als -70 dBm ist gut, schlechter als -80 dBm wird zäh (viele Blockwiederholungen).

Das Abfrageintervall von 300 s ist von diesen Zeiten weit entfernt – die Übertragung lastet das System nur zu wenigen Prozent aus. Für Testzwecke kann das Intervall problemlos auf 10 s gesenkt werden; ist es kürzer als die Übertragungsdauer, holt das Programm einfach lückenlos Bild um Bild.

3.3 Warum überhaupt Modbus?

Ein HTTP-Streaming-Sketch wäre schneller. Modbus TCP wurde trotzdem bewusst gewählt: Das Protokoll ist dasselbe wie bei den übrigen Laboraufbauten (ADAM-Module, Waveshare-Module, ESP32-Slaves, Trendows), die Übertragung ist durch die Register-Semantik vollständig deterministisch, jedes Byte ist CRC-gesichert, und derselbe Sketch lässt sich unverändert von Trendows oder jedem anderen Modbus-Master abfragen. Für ein Bild alle 5 Minuten ist Geschwindigkeit schlicht kein Kriterium.

4 Namensvergabe: die Kamera ohne feste IP finden

4.1 Das Problem

Nach jeder Solar-Abschaltung startet die Kamera neu und bezieht ihre IP-Adresse per DHCP von der Fritz!Box. In der Regel bekommt sie dieselbe Adresse wieder – garantiert ist das aber nicht, insbesondere nach längeren Ausfällen oder einem Router-Neustart. Eine fest im Python-Programm eingetragene IP kann also ins Leere laufen. Eine DHCP-Reservierung in der Fritz!Box würde das lösen, ist hier aber bewusst **nicht** vorausgesetzt.

4.2 Die Lösung: zwei Namen, ein Gerät

Der Sketch meldet die Kamera unter dem Namen **tomatencam** gleich auf zwei Wegen im Netz an:

1. **DHCP-Hostname** (`WiFi.setHostname("tomatencam")`, vor `WiFi.begin()` gesetzt): Die Kamera nennt der Fritz!Box bei jeder Anmeldung ihren Namen. Die Fritz!Box trägt ihn automatisch in ihren lokalen DNS ein – die Kamera ist ab dann als **tomatencam.fritz.box** auflösbar, von jedem Gerät im Heimnetz, ganz ohne Konfiguration am Router.
2. **mDNS** (durch `ArduinoOTA.setHostname("tomatencam") + ArduinoOTA.begin()`): Die Kamera beantwortet Multicast-DNS-Anfragen selbst und ist als **tomatencam.local** erreichbar. Auf diesem Weg findet auch die Arduino IDE den OTA-Netzwerk-Port ("tomatencam at 192.168.x.xxx" – dort ist die aktuelle IP übrigens jederzeit ohne Kabel ablesbar).

Im Python-Programm steht deshalb keine IP mehr, sondern:

```
ESP32CAM_IP = "tomatencam.fritz.box"
```

Entscheidend ist, **wann** aufgelöst wird: `socket.create_connection()` löst den Namen **bei jedem Verbindungsaufbau neu** auf. Nach jedem Verbindungsverlust (und damit nach jeder Solar-Abschaltung) fragt das Programm die Fritz!Box also erneut nach der aktuellen Adresse. Die Kette

Neustart → DHCP (neue IP + Name) → DNS-Eintrag aktualisiert → Dashboard löst neu auf → verbunden

schließt sich vollautomatisch. Sollte `tomatencam.fritz.box` in einem fremden Netz (anderer Router) nicht funktionieren, ist `tomatencam.local` der Ausweichname; als letzte Rückfallebene kann weiterhin eine nackte IP eingetragen werden.

5 Die Stromabschaltung: Erkennung und sicherer Wiederanlauf

5.1 Was bei der Abschaltung passiert

Der Solar-Laderegler trennt die Versorgung bei 10 % Restladung **hart** – für die Kamera ist das wie Netzstecker ziehen. Dabei geht verloren:

- die laufende TCP-Verbindung zum Dashboard,
- die Bildnummer (Register 0 beginnt nach Neustart wieder bei 1),
- der Blitz-Zustand (Register 9 steht wieder auf 0),
- die Betriebszeit (Register 11/12 beginnt wieder bei 0),
- ggf. die IP-Adresse (Kapitel 4).

Nicht verloren gehen: der Sketch selbst (Flash-Speicher), die Referenz und alle Messdaten – die liegen auf dem PC. Ein Datenverlust durch die Abschaltung ist ausgeschlossen, weil die Kamera nichts speichert; schlimmstenfalls entfällt das Bild, das gerade übertragen wurde (die CRC-Prüfung verwirft es, und der nächste Takt holt ein neues).

5.2 Wie das Dashboard den Ausfall erlebt

Während die Kamera aus ist, scheitert jeder Abrufversuch. Die Abfrageschleife fängt **jede** Störung nach demselben Muster ab:

1. Fehler registrieren, Meldung im Dashboard anzeigen ("Störung: ..., nächster Versuch läuft"), Fehlbild-Zähler erhöhen.
2. Socket schließen und auf None setzen – kein halbtoter Zustand.
3. Zwei Sekunden warten, dann regulär im Takt weiterversuchen.

Das Dashboard bleibt dabei voll bedienbar und zeigt das letzte gültige Bild samt Kennwerten. Ein tage- oder wochenlanger Ausfall ist für das Programm derselbe Fall wie ein kurzer WLAN-Schluckauf – es gibt keinen Zustand, aus dem es nicht von allein zurückfindet, und es muss nie neu gestartet werden.

5.3 Der Wiederanlauf, Schritt für Schritt

Sobald der Akku 50 % erreicht, schaltet der Laderegler die Versorgung wieder ein. Dann läuft folgende Kette ab, ohne jeden Eingriff:

1. Kamera bootet, initialisiert die Kamera-Hardware, verbindet sich ins WLAN, meldet ihren Namen an (DHCP + mDNS), startet OTA und den Modbus-Server. Der WLAN-Watchdog im Sketch prüft danach dauerhaft alle 5 s die Verbindung.
2. Beim nächsten Takt der Abfrageschleife gelingt der Verbindungsaufbau (Namensauflösung liefert die aktuelle IP).
3. Direkt nach dem Verbinden stellt das Dashboard den **Blitz-Zustand** wieder her (Register 9 wird mit dem letzten Sollwert beschrieben) – ein eingeschalteter Blitz bleibt aus Nutzersicht also einfach an.
4. Das Dashboard liest mit jedem Bild die **Betriebszeit** (Register 11/12). Ist der neue Wert **kleiner** als der zuletzt gesehene, kann das nur einen Grund haben: Die Kamera wurde zwischenzeitlich stromlos. Das Dashboard schreibt dann "Kamera-Neustart erkannt (Solar-Abschaltung?)" in die Meldungsliste. So sind alle Solar-Zyklen im Dashboard dokumentiert, ohne dass die Kamera dafür eine Uhr bräuchte.
5. Bilder werden mit **Zeitstempel im Dateinamen** gespeichert (bild_20260807_183005_00042.jpg). Dass die Bildnummer nach dem Neustart wieder bei 1 beginnt, ist damit unschädlich – nichts wird überschrieben, und die zeitliche Reihenfolge der Dateien stimmt immer.
6. Die Bewertung läuft mit der **unveränderten Referenz** weiter, denn die liegt in pflanzen_referenz.json auf dem PC. Einzige Empfehlung: Nach sehr langen Ausfällen (Tage) einmal prüfen, ob sich Lichtsituation oder Pflanze stark verändert haben, und die Referenz beim nächsten Gießen ohnehin neu setzen.

5.4 Zeitstempel: warum der PC stempelt und nicht die Kamera

Die ESP32-CAM besitzt keine Echtzeituhr, und selbst eine per NTP gestellte Uhr wäre nach jeder Solar-Abschaltung zunächst wieder falsch. Deshalb stempelt der **PC** beim Empfang Datum und Uhrzeit unten als schwarzen Balken ins Bild – zusammen mit Bildnummer, RSSI und Kamera-Betriebszeit, z. B.:

07.08.2026 18:30:05 Bild 42 | -61 dBm | Kamera an seit 1 h 23 min

Da zwischen Aufnahme und Empfang nur Sekunden liegen, ist der PC-Stempel praktisch exakt. Wichtig: Gestempelt wird nur die **Anzeige- und Speicherfassung** des Bildes; die Bildauswertung (Kapitel 6) läuft immer auf dem unveränderten Original, damit der Balken die Kennwerte nicht verfälscht.

5.5 Software-Updates ohne Kabel (OTA)

Da die Kamera draußen montiert ist, wäre jedes Update per USB-Kabel ein Ärgernis – zumal der serielle Upload bei der ESP32-CAM erfahrungsgemäß störanfällig ist. Der Sketch enthält deshalb ArduinoOTA: Die Kamera erscheint in der Arduino IDE unter Werkzeuge -> Port als Netzwerk-Port **“tomatencam at”**, und der Upload läuft komplett über WLAN.

Das Verfahren ist gegen Stromausfälle **während des Updates** gesichert: Der Flash enthält zwei Programm-Partitionen (Partition Scheme “... with OTA”). Das neue Programm wird in die **inaktive** Partition geschrieben, während das alte weiterläuft. Erst wenn die Übertragung vollständig und die Prüfsumme korrekt ist, wird der Boot-Vermerk umgeschaltet; der Bootloader startet ab dann das neue Programm. Bricht das Update ab – auch durch die Solar-Abschaltung mitten im Vorgang – bleibt der Vermerk auf der alten Partition und die Kamera läuft unverändert weiter. Ein “Zerflashen” über OTA ist damit ausgeschlossen.

Zwei Regeln dazu:

- Jeder künftige Sketch für dieses Board muss die drei OTA-Zeilen wieder enthalten (`#include <ArduinoOTA.h>`, `ArduinoOTA.setHostname(...)` + `ArduinoOTA.begin()` im `setup()`, `ArduinoOTA.handle()` im `loop()`) – sonst ist der Netzwerk-Port nach dem Update weg und es muss einmalig wieder das Kabel heran.
- Der serielle Monitor funktioniert über den Netzwerk-Port nicht; Konsolenausgaben gibt es nur am Kabel. Im Alltag ist das unerheblich, weil alle Betriebsdaten im Dashboard stehen.

6 Die Bildauswertung: von Pixeln zur Gieß-Empfehlung

6.1 Schritt 1: Die Pflanze vom Hintergrund trennen (Excess-Green-Index)

Grundlage ist der in der Agrar-Bildverarbeitung etablierte **Excess-Green-Index**. Für jedes Pixel mit den Farbkanälen R , G , B (je 0 bis 255) wird berechnet:

$$\text{ExG} = 2G - R - B$$

Die Idee: Blattgrün hat einen deutlich höheren Grünkanal als Rot- und Blaukanal zusammen im Vergleich zu neutralen Flächen. Für Grautöne (Erde, Topf, Wand) gilt $R \approx G \approx B$, also $\text{ExG} \approx 0$; für sattes Blattgrün wird ExG deutlich positiv. Ein Pixel zählt zur Pflanze, wenn zwei Bedingungen erfüllt sind:

$$\text{ExG} > 40 \quad \text{und} \quad G > 60$$

Die erste Schwelle (EXG_SCHWELLE) trennt Grün von Neutral, die zweite (MIN_GRUEN) unterdrückt dunkles Rauschen, das sonst bei schwachem Licht als “grün” durchrutschen könnte. Das Ergebnis ist die **Grünmaske** – im Dashboard über das Häkchen “Grünmaske zeigen” einsehbar: Pflanzenpixel bleiben farbig, alles andere wird abgedunkelt. Dieser Kontrollblick ist das wichtigste Werkzeug bei der Inbetriebnahme (stimmt die Maske nicht, stimmen auch die Kennwerte nicht).

6.2 Schritt 2: Zwei Kennwerte je Bild

Aus der Maske werden zwei Zahlen gewonnen:

- **Grünfläche** A : der Anteil der Pflanzenpixel am Gesamtbild in Prozent. Hängen die Blätter durch, verkleinert sich ihre von der Kamera gesehene Projektionsfläche – A sinkt.
- **Schwerpunkt** y_s : die mittlere Höhenlage der Pflanzenpixel, ausgedrückt in Prozent der Bildhöhe (0 = oben, 100 = unten). Berechnet als gewichtetes Mittel der Zeilenkoordinaten:

$$y_s = \frac{\sum_y y \cdot m(y)}{\sum_y m(y)} \cdot \frac{100}{H}$$

mit $m(y)$ = Anzahl Pflanzenpixel in Bildzeile y und H = Bildhöhe. Welkende, herabhängende Blätter verlagern Blattmasse nach unten – y_s **steigt** (größere Werte bedeuten “weiter unten”).

Zusätzlich wird die mittlere Bildhelligkeit bestimmt. Liegt sie unter der Schwelle (MIN_HELLIGKEIT = 40), ist es Nacht oder zu dunkel – die Ampel geht auf GRAU und es findet **keine** Bewertung statt, denn ohne Licht sind die Kennwerte bedeutungslos. Ebenso GRAU: Grünanteil unter 2 % (“Pflanze nicht erkannt”).

6.3 Schritt 3: Glättung

Einzelbilder streuen (Wind bewegt Blätter, Belichtungsautomatik pumpt). Deshalb wird nicht der Momentanwert bewertet, sondern der **Median der letzten 5 Messungen** beider Kennwerte. Der Median ist hier dem Mittelwert überlegen, weil ein einzelner Ausreißer (Vogel im Bild, Wolkenschatten) ihn gar nicht beeinflusst. Bei 5-Minuten-Takt bewertet die Ampel also das Pflanzenverhalten der letzten knapp halben Stunde – für den Wasserhaushalt einer Tomate genau die richtige Zeitskala.

6.4 Schritt 4: Vergleich mit der Referenz und Ampel

Beim Klick auf “Referenz setzen” (direkt nach dem Gießen!) merkt sich das Programm die aktuellen Medianwerte als A_{ref} und $y_{s,\text{ref}}$ in pflanzen_referenz.json. Danach werden je Messung zwei Abweichungen gebildet:

$$\Delta A = \max\left(0, 100 \cdot \left(1 - \frac{A}{A_{\text{ref}}}\right)\right) \quad \Delta y = \max(0, y_s - y_{s,\text{ref}})$$

ΔA ist der **Flächenabfall** in Prozent der Referenzfläche, Δy das **Absinken** des Schwerpunkts in Prozentpunkten der Bildhöhe. Beide sind bei 0 gedeckelt – eine Pflanze, die “besser als die Referenz” dasteht, wird schlicht als GRÜN gewertet. Die Ampel schaltet nach festen Schwellen, wobei jeweils der **schlechtere** der beiden Kennwerte entscheidet:

Bewertung	Bedingung
ROT	$\Delta A \geq 15\%$ oder $\Delta y \geq 4\%$
GELB	$\Delta A \geq 8\%$ oder $\Delta y \geq 2\%$
GRÜN	sonst

Zwei unabhängige Kennwerte machen die Erkennung robuster: Ein reiner Flächenabfall kann auch durch eine Lichtänderung entstehen, ein reines Absinken auch durch Wachstum einer einzelnen Ranke – treten beide gemeinsam auf oder einer deutlich, ist Welke die wahrscheinlichste Erklärung.

Das Verlaufsdiagramm im Dashboard zeigt beide Größen über die letzten zwei Tage (576 Punkte bei 5-Minuten-Takt) mit den ROT-Schwellen als gestrichelte Linien – man sieht der Pflanze also beim “Schlappwerden” zu und erkennt auch das typische Muster nach dem Gießen: Beide Kurven fallen binnen Stunden wieder Richtung null.

6.5 Grenzen des Verfahrens

Das Verfahren ist ein Vergleichsverfahren – es steht und fällt mit der Vergleichbarkeit der Bilder:

- **Kamera fest montieren.** Jede Verschiebung des Bildausschnitts verändert Fläche und Schwerpunkt und entwertet die Referenz.
- **Beleuchtung möglichst gleichmäßig.** Direkte Sonne mit harten Wanderschatten ist der größte Störfaktor. Abhilfe: Standort mit diffusem Licht, notfalls Blitz zuschalten und Referenz mit Blitz setzen.
- **Referenz konsequent nach jedem Gießen neu setzen**, möglichst zur gleichen Tageszeit bzw. bei gleicher Beleuchtung wie die Vergleichsbilder.
- Nachts gibt es systembedingt keine Bewertung (GRAU) – Lücken im Verlaufsdiagramm sind normal, ebenso während der Solar-Abschaltungen.

Alle Messwerte werden zusätzlich in `pflanzen_log.csv` protokolliert (Semikolon-getrennt: Zeit, Bildnummer, Helligkeit, Fläche, Schwerpunkt, Flächenabfall, Absinken, Status) – die Datei lässt sich direkt in Excel öffnen, etwa um die Schwellen anhand der ersten Trockenzyklen nachjustieren.

7 Bedienungsanleitung

7.1 Einmalige Einrichtung

7.1.1 Kamera flashen

1. Arduino IDE öffnen, `ESP32_CAM_Modbus_Snapshot.ino` laden; oben im Sketch WLAN-Name und -Passwort kontrollieren.
2. Unter **Werkzeuge** einstellen (die Einträge erscheinen erst nach der Board-Wahl):
 - Board: **ESP32 Dev Module** (hat garantiert alle Menüpunkte)
 - Upload Speed: **115200** (höhere Raten brechen erfahrungsgemäß ab)
 - Partition Scheme: **Minimal SPIFFS (1.9MB APP with OTA/...)** – entscheidend ist der Zusatz “with OTA”; die KB-Angabe dahinter ist egal
 - PSRAM: **Enabled** (sonst nur reduzierte Auflösung!)
 - Port: der COM-Port des USB-Adapters
3. Hochladen. Bricht der Upload ab: stabile 5-V-Versorgung sicherstellen (nicht der 3,3-V-Pin des Adapters!), kurzes gutes USB-Kabel, notfalls Stützkondensator 470 bis 1000 μF an 5V/GND. Dieses erste Flashen ist das letzte per Kabel – danach geht alles über WLAN.
4. Seriellen Monitor öffnen (115200 Baud), Reset drücken und die Startmeldungen prüfen. Erwartet werden nacheinander: PSRAM gefunden, Kamera initialisiert, WLAN verbunden mit IP-Angabe, OTA bereit (Netzwerk-Port 'tomatencam'), Modbus TCP auf Port 502.

7.1.2 PC vorbereiten

5. Benötigte Pakete: `pip install numpy pillow`
6. In `pflanzen_dashboard.py` prüfen: `ESP32CAM_IP = "tomatencam.fritz.box"`. Zum schnellen Testen darf `INTERVALL` vorübergehend auf 10.0 stehen; für den Dauerbetrieb wieder 300.0 eintragen.

7.1.3 Montage

7. Kamera fest montieren (Stativ/Halterung), Pflanze möglichst bildfüllend, Blickrichtung so, dass die Pflanze frei vor ruhigem Hintergrund steht. Möglichst gleichmäßige Beleuchtung.
8. An die Solarversorgung anschließen. Fertig – an der Fritz!Box ist nichts einzustellen.

7.2 Täglicher Betrieb

7.2.1 Starten

python pflanzen_dashboard.py

Browser: **http://localhost:8083** (die Adresse für Handy/Tablet im selben Netz nennt die Konsole beim Start). Oben links zeigt der Statuspunkt die Verbindung; nach spätestens einem Intervall erscheint das erste Bild mit Zeitstempel-Balken.

7.2.2 Referenz setzen - der wichtigste Handgriff

1. Tomate gießen.
2. Einige Messungen abwarten (im Normaltakt ca. 25 Minuten, denn die Glättung braucht 5 Bilder; im 10-s-Testtakt reicht eine Minute).
3. Im Dashboard **“Referenz setzen”** klicken. Die Bestätigung nennt den Zeitpunkt; er wird dauerhaft unter der Ampel angezeigt.

Diesen Handgriff **nach jedem Gießen wiederholen** - er sagt dem System “so sieht die gut versorgte Pflanze aus”.

7.2.3 Die Dashboard-Elemente

Element	Bedeutung
Ampel mit Statuszeile “Referenz setzen”	Gesamtbewertung (Kapitel 6.4); ROT blinkt speichert den Ist-Zustand als frisch-gegossen-Referenz
Kamerabild / Häkchen “Grünmaske zeigen”	Livebild mit Zeitstempel bzw. Kontrollansicht der Pflanzenerkennung
Kennwerte	Grünfläche, Schwerpunkt, Abweichungen zur Referenz, Helligkeit
Übertragung	Bildnummer, Größe, Dauer, Wiederholungen, verworfene Bilder
Kamera an seit / WLAN-Signal / freier Speicher	Betriebsdaten aus den Registern 11-14; “Kamera an seit” springt nach jeder Solar-Abschaltung zurück
Blitz-Knopf	schaltet die weiße LED der Kamera (überlebt Neustarts)
Verlaufsdigramm	Flächenabfall und Absinken über ca. 2 Tage, Gieß-Schwellen gestrichelt
Meldungen	letzte 10 Ereignisse, u. a. “Kamera-Neustart erkannt (Solar-Abschaltung?)”

7.2.4 Dateien, die das Programm anlegt

Datei/Ordner	Inhalt
pflanzen_log.csv	alle Messwerte, Semikolon-getrennt, Excel-tauglich
pflanzen_bilder/	jedes Bild als JPEG mit Zeitstempel im Namen und im Bild
pflanzen_referenz.json	die aktuelle Referenz (übersteht Programm-Neustarts)

7.3 Software-Update der Kamera (über WLAN)

1. Python-Dashboard mit Strg+C beenden (es hält sonst die einzige Modbus-Verbindung – das stört zwar OTA nicht, aber es ist sauberer, die Kamera in Ruhe zu lassen).
2. Arduino IDE: Werkzeuge -> Port -> **“tomatencam at ...”** wählen.
3. Hochladen. Die Kamera startet danach von selbst neu.
4. Dashboard wieder starten.

Erscheint der Netzwerk-Port nicht: Ist die Kamera gerade in der Solar-Abschaltung? PC im selben Netz? Notfalls Port-Menü erneut öffnen oder IDE neu starten.

7.4 Fehlerbehebung

Symptom	Ursache und Abhilfe
Dauerhaft “Störung: ...” im Dashboard	Kamera stromlos (Solar-Abschaltung – einfach warten) oder Name nicht auflösbar: tomatencam.local bzw. IP eintragen; prüfen, ob ein zweites Programm die einzige Modbus-Verbindung belegt
“Kamera meldet Status 3”	Kamerafehler bei der Aufnahme; tritt er wiederholt auf: Stromversorgung prüfen (Spannungseinbruch), Kamera einmal stromlos machen
“Pflanze nicht erkannt”	Pflanze zu klein im Bild oder zu dunkel: näher heran, Blitz einschalten, ggf. EXG_SCHWELLE senken (z. B. 30)
Ampel springt hin und her	wechselnde Beleuchtung: Standort/Beleuchtung vergleichmäßigen; Referenz bei repräsentativem Licht setzen; ggf. GLAETTUNG erhöhen
Ampel dauernd GELB/ROT trotz gegossener Pflanze Bilder kommen sehr langsam, viele Wiederholungen Serieller Upload bricht ab	Referenz veraltet (Pflanze gewachsen, Kamera verrutscht, Jahreszeit): neu gießen und Referenz neu setzen schwaches WLAN am Standort: RSSI im Dashboard prüfen (schlechter als –80 dBm ist kritisch), Antenne/Position optimieren
“numpy/Pillow fehlen”	Upload Speed 115200? 5-V-Versorgung stabil? Stützkondensator; besser gleich OTA verwenden pip install numpy pillow; ohne diese Pakete läuft nur die Bildübertragung ohne Auswertung

7.5 Einstellparameter im Überblick

Alle Stellschrauben stehen am Anfang von pflanzen_dashboard.py:

Parameter	Standard	Bedeutung
ESP32CAM_IP	tomatencam.fritz.box	Name oder IP der Kamera
INTERVALL	300 s	Zeit zwischen zwei Bildern
EXG_SCHWELLE	40	ab diesem ExG-Wert gilt ein Pixel als grün
MIN_GRUEN	60	Mindest-Grünkanal gegen Dunkelrauschen
MIN_FLAECH	2 %	darunter “Pflanze nicht erkannt”
MIN_HELLIGKEIT	40	darunter “zu dunkel”, keine Bewertung
GELB_FLAECH / ROT_FLAECH	8 % / 15 %	Ampelschwellen Flächenabfall

Parameter	Standard	Bedeutung
GELB_ABSINKEN / ROT_ABSINKEN	2 % / 4 %	Ampelschwellen Schwerpunkt-Absinken
GLAETTUNG	5	Median über die letzten n Messungen
VERLAUF_MAX	576	Punkte im Verlaufsdiagramm (2 Tage bei 5 min)
ORDNER	pflanzen_bilder	Bildablage (leer = nicht speichern)

Im Sketch: BILD_GROESSE (FRAME_SIZE_VGA), BILD_QUALITAET (10) sowie die WLAN-Zugangsdaten.

8 Zusammenfassung

Das System verbindet bewährte Bausteine zu einem wartungsarmen Ganzen: Die solarbetriebene ESP32-CAM liefert als Modbus-TCP-Slave CRC-gesicherte Bilder in 240-Byte-Blöcken (einige Sekunden je VGA-Bild, begrenzt durch die WLAN-Umlaufzeit, nicht durch die Datenmenge). Das PC-Dashboard trennt die Pflanze per Excess-Green-Index vom Hintergrund, verdichtet jedes Bild auf Grünfläche und Blatt-Schwerpunkt, glättet per Median und bewertet gegen eine nach dem Gießen gesetzte Referenz – als Ampel, als Verlaufsdiagramm und als CSV-Protokoll.

Die harten Stromabschaltungen des Solar-Ladereglers sind konstruktiv eingeplant: Namensauflösung statt fester IP (tomatencam.fritz.box, bei jedem Verbindungsaufbau neu), Zeitstempel vom PC statt von der uhrenlosen Kamera, Dateinamen mit Zeitstempel gegen das Zurückspringen der Bildnummer, automatische Wiederherstellung des Blitz-Zustands, Erkennung jedes Neustarts über das Betriebszeit-Register und ein Updateweg (OTA), der selbst bei Stromausfall mitten im Flashen nichts beschädigt. Der einzige regelmäßige Handgriff des Betreibers bleibt der, um den es eigentlich geht: gießen – und danach “Referenz setzen”.